

I. PERFMLOPS: ZERO-HUMAN IN THE LOOP PERFORMANCE TESTING FRAMEWORK - FRAMEWORK DESIGN

A. System Architecture

This framework includes triggering load tests on JMeter as soon as the Python API is deployed after the change. Some performance issues were intentionally induced in the JMeter test script to evaluate the framework's ability to identify bottlenecks. Engineers receive an email after the successful deployment and execution of the JMeter load test. The events generated from JMeter were automatically fed to Good Analytics and then to Google's BigQuery for further analysis. Engineers receive a daily report on the events captured in Google Analytics via email. During the initial setup, the BigQuery ML Model "Boosted tree Classifier" was trained, and the model was later used to identify the number of bottlenecks and their root causes. Google's cloud function service was created to send an email summarizing the tests with bottlenecks and root causes identified by the ML model. These steps are represented in one figure, as shown in Fig. 1.

B. Implementation of Python API

The application layer is built as a RESTful API using Python's Flask framework. This API has a dual-purpose design: a functioning CRUD service for structured data and a performance simulation environment that replicates realistic system characteristics. This design's modularity makes it ideal for loading structured event data into Google Analytics and then exporting it to BigQuery for predictive modeling using SQL ML.

This API has two main functionalities:

- 1) Basic CRUD operations on an in-memory dataset.
 - 2) Performance testing endpoints are designed to simulate various server behaviors.
- Health Check Endpoint
 - "/" → Returns a simple JSON response indicating that the API is active. This is primarily used to monitor the API's availability.
 - CRUD Endpoints

The API simulates a lightweight in-memory database (items =) and provides the standard Create, Read, Update, Delete (CRUD) functionality:

 - POST /items/ → Creates a new item, ensuring unique names.
 - GET /items/"name" → Retrieves an item by name, returning 404 if not found.
 - GET /items/"name" → Retrieves an item by name, returning 404 if not found.
 - PUT /items/"name" → Updates an existing item with new data.
 - DELETE /items/"name" → Deletes an item by name.

This design offers a simple yet extensible data interaction model suitable for experimentation and integration with downstream services.

- Performance Testing Endpoints

To evaluate system resilience and simulate different conditions, the API provides specialized endpoints:

 - /fast → Returns an immediate response, representing minimal latency.
 - /slow → Introduces a 2-second delay before responding, simulating a slow endpoint.
 - /random → Introduces a random delay (0.1–3.0s) to emulate unpredictable response times.
 - /unstable → Has a 20% probability of failure (returns HTTP 500) and otherwise succeeds, modeling unstable network or server conditions.
- Deployment This API was deployed on the local machine using the command (python3 can installed using pip install flask)

```
python3 Python_Code_API_JMeter.py
```

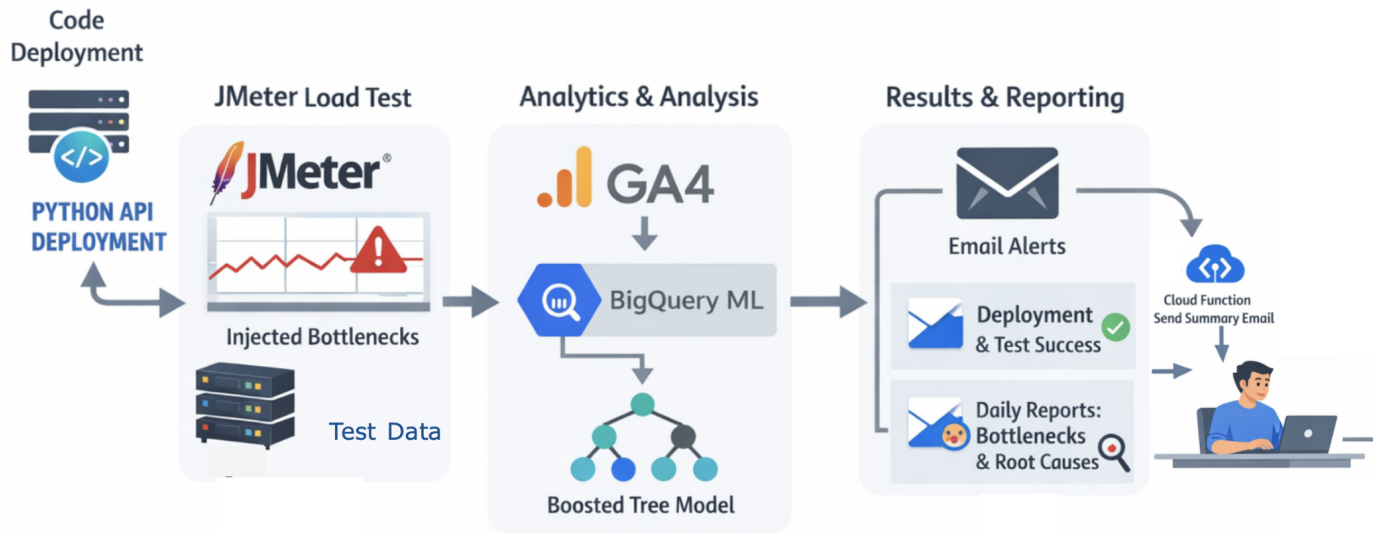


Fig. 1. System Architecture Used in this Study

C. JMeter Script

This JMeter script was designed to perform CRUD operations: Add, Get, Update, and Delete items using the "HTTP Request" and "HTTP Header Manager" elements, which support 50 TPS for 1 hour. Test data for the script was injected using CSV Data Set Config element. Induced response times and response codes to Google Analytics using JSR223 requests for pre/post-processing. A GA request was created to send event data; Google Analytics results are collected as listeners for analysis.

D. Google Analytics 4 Setup

To setup Google Analytics 4, Data stream needs to be created to get measurement ID and API secret as shown in figure below. in this study, stream for a dummy website "https://example.com" was created.

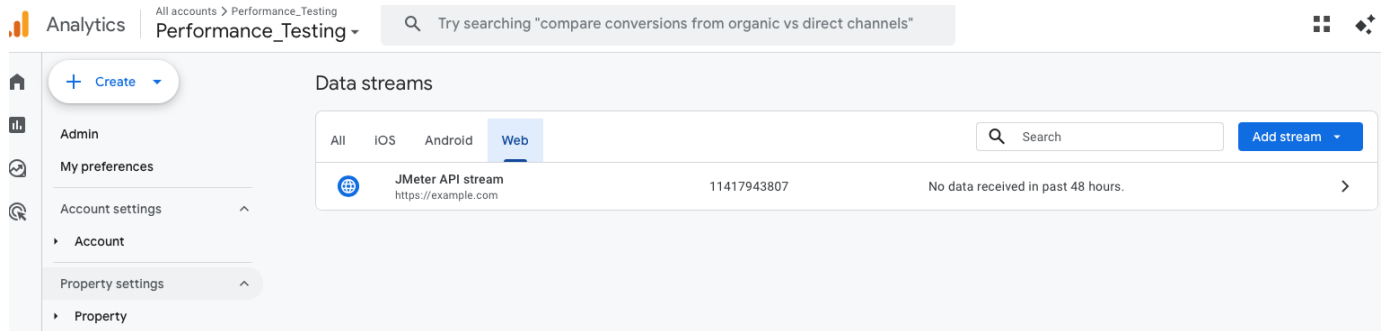


Fig. 2. GA4 Setup

Measurement Id and API Secret can be obtained as shown in figure below:

Web stream details

Web stream details

Stream details

STREAM NAME	STREAM URL	STREAM ID	MEASUREMENT ID
JMeter API stream	https://example.com	[REDACTED]	[REDACTED]

Events

- Enhanced measurement** (Enabled)
 - Automatically measure interactions and content on your sites in addition to standard page view measurement. Data from on-page elements such as links and embedded videos may be collected with relevant events. You must ensure that no personally-identifiable information will be sent to Google. [Learn more](#)
 - Measuring: Page views, Scrolls, Outbound clicks, + 4 more
- Modify events**
 - Modify incoming events and parameters. [Learn more](#)
- Create custom events**
 - Create new events from existing events. [Learn more](#)
- Measurement Protocol API secrets** (Circled in blue)
 - Create an API secret to enable additional events to be sent into this stream through the Measurement Protocol. [Learn more](#)

Fig. 3. Measurement ID and API Secret

Integrated JMeter to send events to GA4 by creating a data stream for the measurement ID and API secret which are passed as part of the API request to feed data into GA from JMeter via API. JMeter sends events to GA using the endpoint (https://www.google-analytics.com/mp/collect?measurement_id=G-XXXXXXX&api_secret=your_secret) and parameters to send in the body of the request along with "event name". Google Analytics then reads the data based on custom events created under the data stream created based on "event name".

Custom events needs to be created as shown in figures below and it takes 12-24hrs for a free tier for the events to be appeared here.

Analytics | All accounts > Performance_Testing

Performance_Testing

Events

Events measure user interactions on your website and app. Key events are tracked for all users and only events from the last 28 days.

[+ Create event](#) [Attribution settings](#) [Custom configuration](#)

Key events **Recent events**

To mark an event as a key event, select the star next to the event.

Fig. 4. First Step in Creating Event

Create an event

Mark as key event
 Key events are the events that are important to your business, such as a purchase or a newsletter signup. They will events and key events report.

Choose how to create an event

☒ **Create without code**
 Create an event that can be detected by your Google tag without adding any code to your website.

Select a data stream where you'd like to track the new event

JMeter API stream

Identify an existing event to use as the trigger for the new event

Event name
 page_view

Matches when
 URL contains

URL*

View more options

Fig. 5. Second Step in Creating Event

Event name should be in line with the name in JMeter script. We can have more conditions under "View more options". We can view more options or update events as shown in the figure below.

+ Create event

Attribution settings

Custom configurations

To mark an event as a key event, select the

Key events

Recent events

Custom events

View all custom events from all streams

Modifications

View and create modifications for existing events or modifications

Fig. 6. View or Update Custom Event

E. BigQuery Setup

After an account has been created in BigQuery, GA can be integrated with the BigQuery project ID using the BigQuery link in the Admin-product link section as shown in figure below. Once linked, Google Analytics automatically creates a dataset in BigQuery and inserts it into the tables.

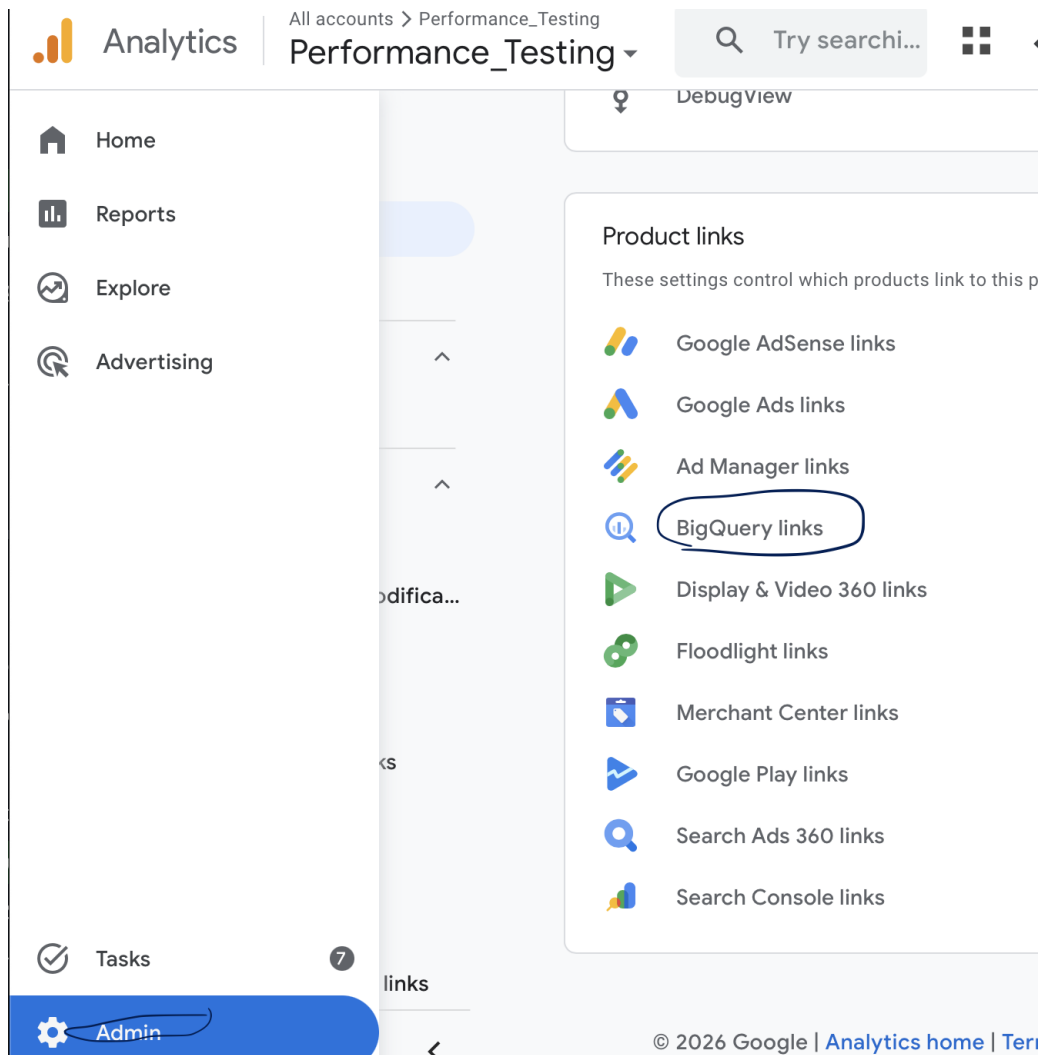


Fig. 7. Big Query Setup in Google Analytics 4

F. BigQuery ML Setup

ML models can be created as shown in figure 8 and trained with simple SQL queries. The process typically starts with preparing the dataset in the BigQuery table and creating the model with the desired model type and method. The regression type of model method is typically used to identify bottlenecks. Based on input features, the target column, and training options, the model is created. Once created, the model is stored inside BigQuery and evaluated in BigQuery. The model can later be used to identify bottlenecks and the root causes of those bottlenecks.

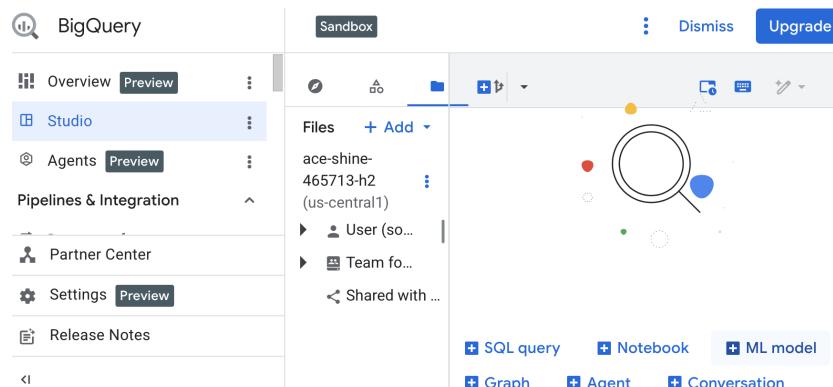


Fig. 8. Create ML Model in BigQuery

Create new ML model

✓ Model name

2 Creation method

3 Model options

Create model

Dataset *
analytics_494976509

Model name *
Performance_Bottlenecks_AI
Provide a name for this model

☒ Save Query
Save an equivalent SQL query for the training options you have chosen (Optional)

Query name *
Performance_Bottlenecks_AI_Query

Region *
us-east4 (Northern Virginia)

Continue

Fig. 9. Create ML Model Step1

Create new ML model

✓ Model name

2 Creation method

3 Model options

Create model

Creation method

☒ Train a model in BigQuery
Train predictive models for forecasting, regression, classification etc. tasks

☐ Connect to Vertex AI LLM service and CloudAI services
Connect to Gemini, 3P and OSS models as service or Cloud AI services such as DocAI

☐ Connect to user managed Vertex AI endpoints
Connect BigQuery to Vertex AI endpoints you manage

☐ Import model
Import trained model artifacts from GCS and run inference in BigQuery

☒ A creation method is required.

Back

Continue

Fig. 10. Create ML Model Step2 and used regression model in this study

This is the final step in creating ML Model. Query used for training can be added under existing query as shown in figure below

Create new ML model

Table / view

Query

Existing query *
Table_query_PT_0830

```

SELECT
  event_timestamp, param.key AS param_key,
  -- Extract response time as double
  MAX(CASE WHEN param.key like '%Response_Time' THEN param.value.double_value END) AS response_t
  -- Extract response code as int
  MAX(CASE WHEN param.key like '%Response_Code' THEN param.value.int_value END) AS response_code
  COUNT(*) AS event_count
FROM
  `ace-shine-465713-h2.analytics_494976509.events_intraday_20250830`,
  UNNEST(event_params) AS param
where param.key <> 'tool'
group by event_timestamp, param_key
order by event_timestamp

```

Select input columns
event_timestamp, param_key, response_ti...

Required options ^

INPUT_LABEL_COLS *
response_time

Optional v

Back

Create model

Fig. 11. Create ML Model final step

G. Cloud Run Function setup

A service was created using Google Cloud Functions, where a Python script was developed to connect to BigQuery, retrieve the query results, and send the summarized output via email. Required dependencies were specified in the requirements.txt file and installed during deployment. Upon successful deployment, the function generated an HTTP endpoint URL, which triggers the execution of the Python code when invoked.